8 Policy Gradient

8.1 Policy Gradient

Basic Idea of Policy Gradient

在 Lecture 7 中我们使用 value function approximation 将 state value 和 action value 从表格 (table) 形式转变为函数 (function) 形式. 事实上 policy π 也是以表格 (如 Table 8.1 所示) 形式呈现，因此同样可以函数化.

State / Action	a_1	a_2	a_3	a_4	a_5
s_1	$\pi(s_1,a_1)$	$\pi(s_1,a_2)$	$\pi(s_1,a_3)$	$\pi(s_1,a_4)$	$\pi(s_1,a_5)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$\pi(s_9,a_1)$	$\pi(s_9,a_2)$	$\pi(s_9,a_3)$	$\pi(s_9,a_4)$	$\pi(s_9,a_5)$

Table 8.1: Action probabilities conditioned on states

记函数化后的 policy 为 $\pi(a|s;\theta)$，其中 $\theta\in\mathbb{R}^d$ 为参数向量，也可以记其为 $\pi_\theta(a,s)$ 和 $\pi_\theta(a|s)$. 常见的做法是使用参数为 θ 的神经网络函数化 $\pi(s,a)$，输入为 s，输出为 $\pi(a_1,s;\theta),\pi(a_2,s;\theta)\ldots$.

Value function approximation 的核心问题之一是将 state value 和 action value 尽量近似准确，以保证后续 policy evaluation 和 policy improvement 足够准确，因此需要最小化目标函数 $J(w)=\mathbb{E}[(v_\pi(S)-\hat{v}(S,w))^2]$ 确保近似效果. 然而当直接优化 policy 时，我们的目标是将当前 policy 不断优化至最优 (optimal) policy. 因此我们需要定义一个 metric / objective function $J(\theta)$ 度量 policy 的好坏，进而定义 optimal policy，并最大化 $J(\theta)$.

Example 8.1. 每次从 s_0 开始一直运行到回合结束，可以将目标函数定义为: $J(\theta)=\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T-1}\gamma^t R_{t+1}\right]=v_{\pi_\theta}(s_0)$. 例如只有两个动作：动作 *1* 得 *10* 分，动作 *2* 得 *0* 分；*policy* 以概率 p_θ 选动作 *1*，那么

$$J(\theta)=10p_\theta+0(1-p_\theta)=10p_\theta. \tag{8.1}$$

此时最大化 J 就是提高好动作的选择概率. 但如果奖励或环境转移未知，我们通常无法直接写出这个具体的 J.

Note 8.1. ① 与 *Lecture 7* 中提到的一样，使用函数形式 *(function representation)* 相较于表格 *(tabular representation)* 的优势在于 *(1) Storage (2) Generalization.* ② *Policy* 的 *tabular* 与 *function representation* 主要有三点不同：

1. 最优策略 π^* 的定义不同: *(1)* 表格形式下 *policy* π^* 满足 $v_{\pi^*}(s)\geqslant v_\pi(s),\forall s$; *(2)* 函数形式下 *policy* π^* 最大化了一个预先定义的函数 *(*作为标量度量*, scalar metric).*
2. 获得某 *action* 的概率 $\pi(s,a)$ 的方式不同: *(1)* 表格形式下直接查表; *(2)* 函数形式下需要重新代入计算一遍.
3. 更新 $\pi(s,a)$ 方式不同: *(1)* 表格形式直接更改相应概率值; *(2)* 函数形式改变参数 θ，间接改变相应概率值.

8.2 Metrics to Define Optimal Policies

Metric 1: Average Value

度量 policy π 的好坏最简单和直观的想法就是对所有的 state value 取加权平均，这样的 metric 被称为 **average state value / average value**:

$$\bar{v}_\pi=\sum_{s\in\mathcal{S}}d(s)v_\pi(s)=\boldsymbol{d}^T\boldsymbol{v}_\pi. \tag{8.2}$$

其中 $\bar{v}_\pi$ 为所有 state value 的加权平均，权重 $d(s)\geqslant 0,\ \forall s,\sum_{s\in\mathcal{S}}d(s)=1$. $\boldsymbol{v}_\pi=[\ldots,v_\pi(s),\ldots]^T\in\mathbb{R}^{|\mathcal{S}|},\boldsymbol{d}=[\ldots,d(s),\ldots]^T\in\mathbb{R}^{|\mathcal{S}|}$. 权重 $\boldsymbol{d}$ 的选择主要分为与 policy 相关和不相关两种情况.

Case 1: Independent of the Policy. 最简单的情况是 d 的选择不依赖于 π，因此也不依赖于参数 θ，这样在优化过程中计算梯度时就不需要对 d 的部分进行计算. 此时将其记为 d_0，不过即使与 π 不相关，d_0 也有不同的选择：

1. 将所有的 state 平等对待，即 $d_0(s)=1/|\mathcal{S}|$;
2. 可能只关心某一个状态 s_0 (例如固定 s_0 处出发)，此时可以赋值 $d_0(s_0)=1,\quad d_0(s\neq s_0)=0$.

Case 2: Independent of the Policy. 第二种情况是 d 的选择依赖于 π. 一个常见的做法是 $d(s)=d_\pi(s)$，即 policy π 的 stationary distribution (满足 $d_\pi^TP_\pi=d_\pi^T$).

Metric 2: Average Reward

度量 π 的另一种方式是对 immediate reward 取加权平均，称之为 **average one-step reward / average reward**:

$$\bar{r}_\pi \triangleq \sum_{s\in\mathcal{S}} d_\pi(s) r_\pi(s) = \mathbb{E}[r_\pi(S)] \tag{8.3}$$

其中 $S \sim d_\pi$ 为 stationary distribution，$r_\pi(s)$ 为 state s 处的 immediate reward，定义为:

$$r_\pi(s) \triangleq \sum_{a\in\mathcal{A}} \pi(a|s) r(s,a), \quad r(s,a) = \mathbb{E}[R|s,a] = \sum_{r,s'} rp(r,s'|s,a). \tag{8.4}$$

在论文中计算 average reward 的常见形式是对于一个给定 policy π，已经生成了一个 trajectory，其中 reward 记为 $(R_{t+1}, R_{t+2}, \ldots)$，此时该 trajectory 的 single-step reward 为:

$$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[R_{t+1} + R_{t+2} + \cdots + R_{t+n} | S_t = s_0\right] = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k} | S_t = s_0\right], \tag{8.5}$$

其中 s_0 为初始 state.

Note 8.2. 对于无穷多步的 *trajectory*，初始位置已不再有意义*:*

$$\lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k} | S_t = s_0\right] = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k}\right] \overset{\text{大数定律}}{=} \sum_s d_\pi(s) r_\pi(s) = \bar{r}_\pi. \tag{8.6}$$

Note 8.3. ① 由于上述的 π 都是被 θ 参数化的，因此定义的 *metric* 都是 θ 的函数. ② 优化上面介绍的两种 *metric* 的效果实际上是一样的，因为在 *discounted case* 下有*:*

$$\bar{r}_\pi = (1-\gamma)\bar{v}_\pi. \tag{8.7}$$

③ 关于 *average value* 有如下结论 *(*证明详见附录*):*

$$J(\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right] = \sum_{s\in\mathcal{S}} d(s)\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} | S_0 = s\right] = \sum_{s\in\mathcal{S}} d(s) v_\pi(s) = \bar{v}_\pi. \tag{8.8}$$

Small Conclusion

两种 metric 都度量了 policy 的好坏，即都定义了相应的目标函数 $J(\theta)$，我们的目标是 $\max J(\theta)$. 事实上这两种 metric 都有两种等价的定义:

Average value:

1. $J(\theta) = \sum_{s\in\mathcal{S}} d(s) v_\pi(s)$
2. $J(\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right]$

Average Reward:

1. $J(\theta) = \sum_{s\in\mathcal{S}} d_\pi(s) r_\pi(s)$
2. $J(\theta) = \lim_{n\to\infty} \frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k} | S_t = s_0\right]$

8.3 Gradients of the Metrics

Introduction

Metric 梯度的计算事实上是 policy gradient methods 中最复杂的部分，因为在 metric 定义中有很多的情况，例如: ① 区分使用的 metric 是 $\bar{v}_\pi, \bar{r}_\pi, \bar{v}_\pi^0$; ② 区分是否是 discounted case. 因此此处仅做简要介绍.

Gradients of the Objectives

下面直接给出策略梯度的公式:

$$\nabla_\theta J(\theta) = \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s,\theta) q_\pi(s,a), \tag{8.9}$$

其中 $J(\theta)$ 可为 $\bar{v}_\pi, \bar{r}_\pi$ 和 $\bar{v}_\pi^0$; $=$ 可以为严格相等 $(=)$、约等 $(\approx)$ 与成比例 $(\propto)$; η 为分布/ state 的权重. 具体形式如下:

$$\nabla_\theta \bar{r}_\pi \simeq \sum_s d_\pi(s) \sum_a \nabla_\theta \pi(a|s,\theta) q_\pi(s,a), \tag{8.10a}$$

$$\nabla_\theta \bar{v}_\pi = \frac{1}{1-\gamma} \nabla_\theta \bar{r}_\pi \quad \text{(in discounted case)}, \tag{8.10b}$$

$$\nabla_\theta \bar{v}_\pi^0 = \sum_{s\in\mathcal{S}} \rho_\pi(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s,\theta) q_\pi(s,a). \tag{8.10c}$$

其中式 (8.10a) 中在 discounted case 下是约等于，在 undiscounted case 下是严格等于.

由于 $\nabla_\theta \ln \pi(a|s,\theta) = \frac{\nabla_\theta \pi(a|s,\theta)}{\pi(a|s,\theta)}$，可以得到如下非常有用的形式:

$$\begin{aligned} \nabla_\theta J(\theta) &= \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a|s,\theta) q_\pi(s,a) = \sum_s d(s) \sum_a \pi(a \mid s,\theta) \nabla_\theta \ln \pi(a \mid s,\theta) q_\pi(s,a) \\ &= \mathbb{E}_{S\sim d}\left[\sum_a \pi(a|S,\theta) \nabla_\theta \ln \pi(a|S,\theta) q_\pi(S,a) \right] = \mathbb{E}\left[\nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A) \right] \approx \nabla_\theta \ln \pi(a|s,\theta) q_\pi(s,a). \end{aligned} \tag{8.11}$$

将梯度 $\nabla_\theta J(\theta)$ 写成期望的形式再用单次采样 $\nabla_\theta \ln \pi(a|s,\theta) q_\pi(s,a)$ 近似是为说明可以使用 *SGD* 进行优化.

Note 8.4. 为保证 $\ln \pi(a|s,\theta)$ 合理，需要保证 $\pi(a|s,\theta) > 0$，因此不能采用 *greedy* 方式. 可以使用 *softmax function* $\pi(a|s,\theta) = \frac{e^{h(s,a,\theta)}}{\sum_{a'\in\mathcal{A}} e^{h(s,a',\theta)}}$，其中 $h(s,a',\theta)$ 为另外的函数，例如神经网络，直接将其输出层定为 *softmax* 即可.

8.4 REINFORCE Algorithm

Gradient-Ascent Algorithm

我们的目的是得到最优 policy，故需要最大化 $J(\theta)$，因此此时需要使用梯度上升算法:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta) = \theta_t + \alpha \mathbb{E}\left[\nabla_\theta \ln \pi(A|S,\theta_t) q_\pi(S,A) \right]. \tag{8.12}$$

在实际使用中一般使用如下随机版本 (stochastic gradient ascent method):

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_\pi(s_t,a_t). \tag{8.13}$$

由于 q_π 实际未知，因此需要与之前算法中一样对其进行估计，得到如下梯度上升算法公式:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_t(s_t,a_t). \tag{8.14}$$

How to do sampling? 在随机算法中我们需要通过采样将期望进行替换，即:

$$\mathbb{E}_{S\sim d, A\sim\pi}\left[\nabla_\theta \ln \pi(A|S,\theta_t) q_\pi(S,A) \right] \longrightarrow \nabla_\theta \ln \pi(a|s,\theta_t) q_\pi(s,a) \tag{8.15}$$

① 对 $S \sim d$ 采样时，d 是 π 的 long run behavior，但实际中一般无法获得 long run behaviour，因为数据收集并不容易，因此收集太多数据并不实际.

② 采样 $A \sim \pi(A|S,\theta)$ 时，a_t 应当来自当前 policy $\pi(\theta_t)$ 下的 state s_t，因此 policy gradient 是 on-policy 方法[a].

[a] 也有 off-policy 的情况，但是需要额外技巧

REINFORCE Algorithm

How to do interpret REINFORCE? 由于 $\nabla_\theta \ln \pi(a_t|s_t,\theta_t) = \frac{\nabla_\theta \pi(a_t|s_t,\theta_t)}{\pi(a_t|s_t,\theta_t)}$，因此式(8.14)可以写为：

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_t(s_t,a_t) = \theta_t + \alpha \underbrace{\left(\frac{q_t(s_t,a_t)}{\pi(a_t|s_t,\theta_t)} \right)}_{\beta_t} \nabla_\theta \pi(a_t|s_t,\theta_t) \triangleq \theta_t + \alpha\beta_t \nabla_\theta \pi(a_t|s_t,\theta_t). \tag{8.16}$$

此时 $\alpha\beta_t$ 作为新的步长. 可以看出实际上式 (8.16) 是在优化 π. 因为可以将 $-\pi(a_t|s_t,\theta_t)$ 看作是目标函数 $f(\theta_t)$. 根据一阶泰勒展开，当 $\theta_{t+1} - \theta_t$ 非常小时有:

$$\pi(a_t|s_t,\theta_{t+1}) \approx \pi(a_t|s_t,\theta_t) + (\nabla_\theta \pi(a_t|s_t,\theta_t))^T (\theta_{t+1} - \theta_t) = \pi(a_t|s_t,\theta_t) + \alpha\beta_t \|\nabla_\theta \pi(a_t|s_t,\theta_t)\|^2. \tag{8.17}$$

因此若 $\beta_t > 0$ 则 $\pi(a_t|s_t,\theta_{t+1}) > \pi(a_t|s_t,\theta_t)$；若 $\beta_t < 0$ 则 $\pi(a_t|s_t,\theta_{t+1}) < \pi(a_t|s_t,\theta_t)$.

Note 8.5. 事实上 $\beta_t = \frac{q_t(s_t,a_t)}{\pi(a_t|s_t,\theta_t)}$ 是在平衡探索 *(exploration)* 和利用 *(exploitation)*.

① ***Exploration:*** 可以看到 β_t 与 $\pi(a_t|s_t,\theta_t)$ 成反比，由于 $\pi(a_t|s_t,\theta_{t+1}) - \pi(a_t|s_t,\theta_t) \approx \alpha\beta_t \|\nabla_\theta \pi(a_t|s_t,\theta_t)\|^2$，因此 $\pi(a_t|s_t,\theta_t)$ 较小 $\to \beta_t$较大 $\to \pi(a_t|s_t,\theta_{t+1})$变大. 因此当前时间步 t 选择某个 *action* 的概率小，那么下一时间步 $t+1$ 选择该 *action* 的概率会变大，因此是一种探索.

② ***Exploitation:*** 可以看到 β_t 与 $q_t(s_t,a_t)$ 成正比，因此 q_t较大 $\to \beta_t$较大 $\to \pi(a_t|s_t)$变大. 因此对于 *action value* 更大的 *action* (q_t 更大)，当前的 *policy* 倾向于选择它 ($\pi_t(a_t|s_t)$) 大，因此是一种利用.

在式(8.14) 中，如果使用了 MC 的方法得到 $q_t(s_t,a_t)$，即采样得到一个 episode，计算相应的 return(g_t)，赋值给 q_t，那么就称其为 **REINFORCE** 算法. REINFORCE 算法是最早也是最简单的 policy gradient 的算法.

Algorithm 14 Policy Gradient by Monte Carlo (REINFORCE)

Initialization: A parameterized function $\pi(a \mid s,\theta)$, $\gamma \in (0,1)$, and $\alpha > 0$.
Aim: Search for an optimal policy maximizing $J(\theta)$.
for the kth iteration **do**
  Select s_0 and generate an episode following $\pi(\theta_k)$. Suppose the episode is $\{s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T\}$.
  ▷ **offline**(because a complete episode is required)
  for $t = 0, 1, \ldots, T-1$ **do**
    Value update:

$$q_t(s_t,a_t) = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$$

    Policy update:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t \mid s_t,\theta_t) q_t(s_t,a_t)$$

  end for
  Set $\theta_k = \theta_T$
end for